UNIVERSIDADE DO ALGARVE
FACULDADE DE CIÊNCIAS E TECNOLOGIA

Algoritmos e Estruturas de Dados

A • Exame Época Normal • 16 de janeiro de 2023 • 16:00 – 18:00

- Para perguntas com resposta de escolha múltipla, respostas erradas com cotação c e n respostas possíveis descontam $-c / (n - 1)$.
- As respostas às perguntas de desenvolvimento deverão ser as mais detalhadas possíveis, indicando os vários passos intermédios da resolução.
- Na mesa em que está a fazer o exame deve ter apenas lápis/caneta, identificação, este enunciado, e folha de teste.
- Identifique cada folha de teste com o seu nome e número, em letra bem legível.

Grupo I: Perguntas de escolha múltipla

1. (0.5) Considere o seguinte código

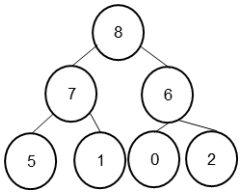
```
public static Integer copia(Integer i)
{
    Integer zero = new Integer(0);
    return new Integer(i.intValue());
}
public static void main(String[] args)
{
    Integer i1,i2,i3,i4,i5;

    i1 = new Integer(12);
    i2 = new Integer(34);
    i3 = copia(i1);
    i4 = copia(i2);
    i1 = null;
    i5 = copia(i3);
    i3 = null;
    ...
}
```

No fim da execução da linha *"i3=null;"*, quantos objetos foram criados no *heap*, e quantos foram marcados para remoção?

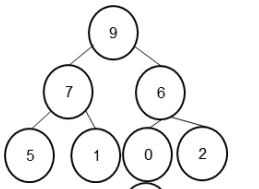
a) 5 criados, 2 marcados para remoção.
b) 8 criados, 2 marcados para remoção.
c) 5 criados, 5 marcados para remoção
d) 8 criados, 5 marcados para remoção

2. (0.5) Considere o seguinte *heap*:

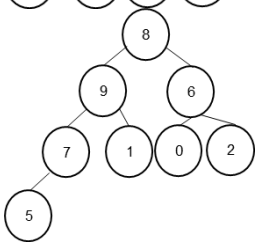


Qual dos seguintes *heaps* corresponde ao resultado de se inserir o elemento 9 no *heap*?

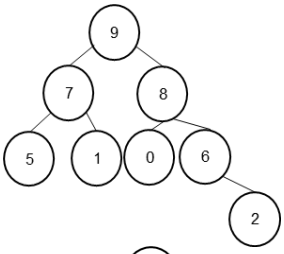
a)



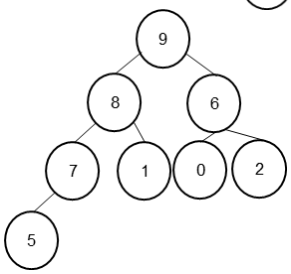
b)



c)



d)



e) Nenhum dos *heaps* apresentados corresponde ao resultado de se inserir o elemento 9.

3. (0.5) Considere o algoritmo de ordenação *MergeSort BottomUp* aplicado ao seguinte *array* [4, 2, 5, 0, 1, 7, 3, 8].
Indique qual das seguintes sequências corresponde a uma sequência válida e pela ordem correta do estado do *array* após as primeiras 2 iterações do ciclo exterior do algoritmo.

a)
[0, 2, 4, 5, 1, 3, 7, 8]
[0, 1, 2, 3, 4, 5, 7, 8]

b)
[0, 2, 4, 5, 1, 7, 3, 8]
[0, 1, 2, 3, 4, 5, 7, 8]

c)
[2, 4, 0, 5, 1, 7, 8, 3]
[0, 2, 4, 5, 1, 3, 7, 8]

d)
[0, 2, 4, 5, 1, 3, 7, 8]
[0, 2, 4, 1, 3, 5, 7, 8]

4. (0.5) Depois de ter implementado um determinado algoritmo, decidiu executar ensaios de razão dobrada para tentar estimar de forma empírica a complexidade temporal assimptótica do algoritmo. Os resultados obtidos foram os seguintes:

i	n	time(ms)	estimated r
…			
8	12800	005,7292	0,000
9	25600	007,8125	1,364
10	51200	016,6667	2,133
11	102400	028,6458	1,719
12	204800	085,4167	2,982
13	409600	228,6458	2,677
14	819200	465,6250	2,036

Indique a complexidade temporal estimada, tendo em conta os resultados obtidos:

a) $O(n^3)$
b) $O(\log n)$
c) $O(1)$
d) $O(n)$
e) $O(n^2)$

5. (0.5) Indique a aproximação tilde mais correta para a seguinte expressão:

$$2n^2 + n\log_2 2^n + 5n + \log_2 n$$

Selecione uma opção de resposta:

a) $\sim 3n^2$
b) $\sim 2n^2$
c) $\sim 5n$.
d) $\sim n \log_2 2^n$

6. (0.5) Considere as seguintes afirmações:
1. Na implementação de uma tabela de dispersão, a escolha de uma boa função de dispersão ajuda a minimizar a existência de colisões.
2. A resolução de colisões através de encadeamento separado consiste em guardar chaves que têm o mesmo valor de dispersão (hash value) no mesmo balde. Quando procuramos por uma chave, temos de pesquisar de forma linear por todas as chaves no mesmo balde.
3. Uma colisão ocorre quando duas chaves diferentes geram o mesmo valor de dispersão (hash value).
4. Uma tabela de encadeamento separado com número de elementos guardados 4 vezes superior ao tamanho *m* da tabela, vai fazer com que em média, tenhamos 4 elementos guardados em cada balde

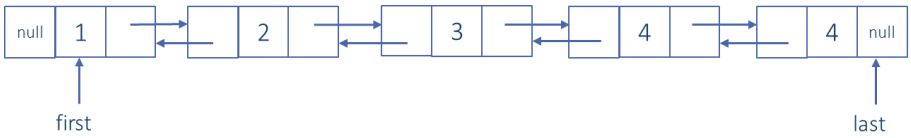
Quais das afirmações acima são verdadeiras?:
a) nenhuma
b) 1, 2 e 3
c) todas
d) 2,3,4
e) 1,3,4
f) 1,2,4

Grupo II: Perguntas de desenvolvimento

7. (2.0) Escreva um método estático chamado máximoImpar na linguagem Java, que dado um array de inteiros a, retorna o máximo inteiro ímpar que exista no array. Se o array recebido não tiver nenhum elemento ímpar, deve ser lançada uma exceção do tipo *IllegalArgumentException*, com uma mensagem de erro apropriada.

Por exemplo, se a=[1,2,3,4,5], então a função devolve 5. Se a=[8,7,5,4,3,2,1] a função devolve 7.

8. (1.0) Tal como lecionado nas aulas teóricas, uma lista ligada corresponde a uma estrutura de dados composta por uma sequência de nós encadeados (em que cada nó tem um ponteiro para o nó seguinte). Existe uma variante desta estrutura de dados que consiste numa lista duplamente ligada (em que cada nó tem um ponteiro para o nó anterior e o próximo nó), como podemos ver na imagem seguinte:



Indique as vantagens e desvantagens de uma lista duplamente ligada face a uma lista ligada simples.

9. (2.0) Considere uma pilha (*stack*) e uma fila (*queue*) inicialmente vazias, sobre as quais é executada a seguinte sequência de instruções:

```
StackList<Integer> s = new StackList<Integer>();
QueueList<Integer> q = new QueueList<Integer>();
int j;

for(int i = 0; i<3; i++)
{
    q.enqueue(i);
    s.push(i);
}
```

```
for(int i = 0; i<3; i++)
{
    q.enqueue(s.pop());
}

for(int i = 0; i<6; i++)
{
    s.push(q.dequeue());
}

while(!s.isEmpty()) System.out.print(s.pop() + "-");
```

Represente a fila q e a pilha s, com os elementos introduzidos e removidos ao longo da execução do código e indique a sequência impressa no ecrã quando o código é executado.

10. (1.5) Considere o Projeto 3 da cadeira. Dois dos métodos pedidos no problema B (SmartMergeSort) correspondiam ao *getNaturalRun*, e *getNaturalOrMakeAscendingRun*. Estes dois métodos recebem 3 argumentos: o array no qual se deve procurar a run; o índice low do array de onde se deve começar a procura; e o índice high que representa a posição do último elemento do array que devemos considerar para a run. Lembre-se que uma run pode ser representada pela notação {start+length} onde start corresponde ao índice onde a run começa, e length ao tamanho da run.

Indique as runs retornadas pelas seguintes invocações:

a)(0.3) getNaturalRun([5,8,10,7,9,11,13],0,6)

b)(0.3) getNaturalRun([5,8,10,7,9,11,13],3,5)

c)(0.3) getNaturalRun([6,5,4,3,2,1],3,5)

d)(0.3) getNaturalOrMakeAscendingRun([6,5,4,3,2,1],0,5)

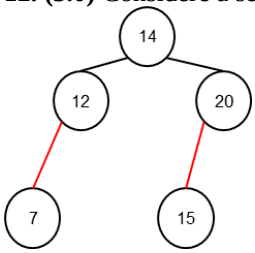
e)(0.3) getNaturalOrMakeAscendingRun([7,7,5,4,2,1],0,5)

11. (2.0) Considere o método *partition* usado no algoritmo de ordenação ***quicksort***, com a assinatura *int[] partition(Comparable[] a, int low, int high)*. Suponha que o método *partition é* invocado com os seguintes argumentos:

a=[5, 7, 3, 8, 1, 6, 4, 9, 2,10], low=0,high=9

Indique os vários passos do método *partition*, indicando o conteúdo do *array* após cada troca de elementos, assumindo que o pivô é escolhido como sendo o elemento da posição *low*. Indique também qual a posição do pivô, retornado pelo método.

12. (3.0) Considere a seguinte árvore rubro-negra (red-black):



a) (1.5) Insira a chave 23 na árvore, desenhando os principais passos intermédios, e indique explicitamente quando efetuar uma das seguintes operações: Rotate Left, Rotate Right, e Flip. Desenhe ligações vermelhas a tracejado e ligações negras com linhas contínuas. Desenhe também a árvore final.

b) (1.5) Insira a chave 5 **na árvore original**, desenhando os principais passos intermédios, e indique explicitamente quando efetuar uma das seguintes operações: Rotate Left, Rotate Right, e Flip. Desenhe ligações vermelhas a tracejado e ligações negras com linhas contínuas. Desenhe também a árvore final.

13. (1.0) As tabelas de símbolos implementadas através de árvores de pesquisa binária (ex: árvore *red-black* ou *treap*) são mais lentas que tabelas de símbolos implementadas usando uma tabela de dispersão. No entanto existem situações onde se escolhe utilizar as primeiras em detrimento das segundas. Explique em que situações isso acontece, e dê um exemplo de um problema onde uma árvore de pesquisa binária seja preferível a uma tabela de dispersão.

14. (2.0) Considere uma tabela de dispersão de dimensão m = 7, com resolução de colisões por dupla dispersão, e funções de dispersão h1(k) = k mod m e h2(k) = 5 – k mod 5

Sabendo que que inicialmente a tabela se encontra preenchida com as chaves indicadas, indique quais os elementos presentes em cada posição do array de chaves e de valores, apos a inserção da sequência de pares <chave , valor> abaixo. **Indique o valor de h1 e h2 calculado para cada chave**. Indique uma posição vazia com ▬.

<3 , “a”>, <8 , “b”>, <10 , “c”>, < 7, “d”>, <9 , “e”> ,

i	0	1	2	3	4	5	6
chaves	7	15					
valores	“k”	“a”					

15. (2.5) Considere o seguinte grafo pesado não dirigido. Considere a execução do algoritmo de Prim sobre o grafo pesado não dirigido da figura abaixo. Qual a sequência pela qual o algoritmo de Prim visita as arestas de corte, assumindo que o algoritmo inicia a procura a partir do **vértice A**? (Em caso de empate, utilize a ordem alfabética dos vértices para desempatar). Desenhe a árvore de cobertura mínima obtida pelo algoritmo.

